

AffineQuantizedObserverBase#

<https://pytorch.org/docs/stable/generated/torch.ao.quantization.observer.Aff>

Description:

Observer module for affine quantization (pytorch/ao) `block_size` (granularity and) – The granularity of the quantization, must specify at least one, if both are specified `block_size` takes precedence Current supported granularity type are `PerTensor` and `PerAxis` args (other) – please see `:class:torchao.dtypes.AffineQuantizedTensor` Calculate quantization parameter based on the stats attached to the observer module and returns a tuple of scale and zero_point Tensor tuple[torch.Tensor, torch.Tensor]

Function Signature

```
class
torch.ao.quantization.observer.AffineQuantizedObserverBase(mapping_type, target_dtype, granularity, quant_min=None,
quant_max=None, eps=None, scale_dtype=None,
zero_point_dtype=None, preserve_zero=True,
zero_point_domain=ZeroPointDomain.INT, **kwargs)[source]#
Parameters
abstract calculate_qparams()[source]#
Return type
convert(model, observer_node)[source]#
```

Returns:

Tensor

Code Examples

```
>>> Foo.with_args = classmethod(_with_args)
>>> foo_builder = Foo.with_args(a=3, b=4).with_args(answer=42)
>>> foo_instance1 = foo_builder()
>>> foo_instance2 = foo_builder()
>>> id(foo_instance1) == id(foo_instance2)
False
```

Detailed Documentation

Documentation

Observer module for affine quantization (pytorch/ao)

`block_size` (granularity and) – The granularity of the quantization, must specify at least one, if both are specified `block_size` takes precedence Current supported granularity type are `PerTensor` and `PerAxis`

`args` (other) – please see :class:torchao.dtypes.AffineQuantizedTensor

Calculate quantization parameter based on the stats attached to the observer module and returns a tuple of scale and zero_point Tensor

tuple[torch.Tensor, torch.Tensor]

Converts the observer node in the graph into its quantized representation

`model` (GraphModule) – graph module to convert the observer node in

`observer_node` (Node) – the observer node to convert

forward function should take the input tensor and updates internal stats and return the original input Tensor

Wrapper that allows creation of class factories.

This can be useful when there is a need to create classes with the same constructor arguments, but different instances. Can be used in conjunction with `_callable_args`

